CodeQuest AI Academy

Complete transformation blueprint and agent handoff package

Existing reach	7,000+ learners
Core curriculum	12 tracks
Problem manifest	10,000 validated-content slots
Books	5 x 100-page production blueprints
Knowledge library	20 optional deep dives

1. Product vision and safeguards

CodeQuest AI Academy evolves the existing coding platform into a coherent beginner-to-advanced AI education product while preserving real users, progress, and the Learn. Build. Improve. identity.

The implementation must begin with a repository and data audit. Migrations are additive and reversible. Existing accounts, learner progress, analytics, and useful coding content are protected. No fake data, placeholder controls presented as complete, unsafe code execution, or unreviewed auto-publishing is allowed.

The curriculum is English-only. It combines short interactive practice with rigorous explanations, real editors, projects, five books, and optional advanced knowledge files.

2. Curriculum architecture

T01 - AI Orientation and Computational Thinking (400 problems)

Explain core AI ideas, decompose problems, and identify responsible uses before writing models.

Modules: What AI Can and Cannot Do; Data Models and Algorithms; Problem Decomposition; Logic Patterns and Pseudocode; Responsible AI Foundations

T02 - Python for AI (1400 problems)

Write, test, debug, and organize Python programs used in data and AI workflows.

Modules: Syntax Variables and Types; Conditions Loops and Functions; Collections and Comprehensions; Files Exceptions and Modules; Object-Oriented Python; Testing Debugging and Complexity; NumPy Foundations; Python Projects for Data and AI

T03 - Algorithms and Data Structures (1000 problems)

Select and implement efficient structures and algorithms, then justify complexity tradeoffs.

Modules: Complexity and Big O; Arrays Strings Stacks and Queues; Hash Tables Sets and Heaps; Linked Structures Trees and Graphs; Sorting Searching and Recursion; Greedy Dynamic Programming and Backtracking; Graph Algorithms; Interview and AI Pipeline Problems

T04 - Mathematics for AI (1800 problems)

Use the mathematical language required to understand, implement, and diagnose learning algorithms.

Modules: Arithmetic Algebra and Functions; Vectors Matrices and Linear Systems; Linear Transformations and Eigen Concepts; Probability Rules and Random Variables; Statistics Estimation and Sampling; Derivatives Gradients and Partial Derivatives; Optimization and Gradient Descent; Information Theory and Numerical Stability

T05 - Data Skills (800 problems)

Transform raw data into documented, reproducible, model-ready datasets.

Modules: NumPy Arrays and Vectorization; Pandas Tables and Joins; SQL for Analysis; Cleaning Missing Values and Outliers; Exploratory Data Analysis; Visualization and Communication; Feature Engineering and Data Leakage

T06 - Machine Learning Foundations (1600 problems)

Build and evaluate classical ML systems without data leakage and with appropriate baselines.

Modules: ML Workflow and Baselines; Linear and Polynomial Regression; Logistic Regression; K-Nearest Neighbors and Naive Bayes; Decision Trees and Random Forests; Support Vector Machines; Clustering and Dimensionality Reduction; Metrics Validation and Imbalanced Data; Pipelines Tuning and Interpretation

T07 - Advanced Machine Learning (700 problems)

Compare advanced methods, tune them responsibly, and reason about uncertainty and failure modes.

Modules: Boosting Bagging and Stacking; Regularization and Generalization; Probabilistic Models; Anomaly Detection and Recommenders; Time Series Forecasting; Causal Thinking and Experiment Design

T08 - Deep Learning (1100 problems)

Implement, train, diagnose, and compare neural models while controlling compute and overfitting.

Modules: Perceptrons and Neural Networks; Forward Pass Loss and Backpropagation; Optimization Initialization and Normalization; Convolutional Neural Networks; Sequence Models and Attention; Transformers; Autoencoders and Generative Models; PyTorch Training and Debugging

T09 - Natural Language Processing and LLMs (500 problems)

Build and evaluate practical language systems without confusing fluent output with correctness.

Modules: Text Processing and Classical NLP; Embeddings and Semantic Search; Transformers and Language Modeling; Prompting Structured Output and Tools; Retrieval-Augmented Generation; LLM Evaluation Safety and Cost

T10 - Computer Vision (300 problems)

Prepare image data, train vision models, and evaluate performance across relevant groups and conditions.

Modules: Image Representation and Augmentation; Classification with CNNs; Detection Segmentation and Transfer Learning; Vision Evaluation and Bias

T11 - Reinforcement Learning (200 problems)

Model sequential decisions and explain the tradeoff between exploration, reward, and safety.

Modules: Agents Environments Rewards and Policies; Bandits and Exploration; Markov Decision Processes; Q-Learning and Policy Gradients

T12 - MLOps Responsible AI and Capstones (200 problems)

Ship documented AI systems with monitoring, risk controls, reproducible experiments, and honest claims.

Modules: Reproducibility Versioning and Experiment Tracking; Serving Monitoring and Drift; Privacy Fairness Security and Documentation; Portfolio Capstone Planning

3. Exact 10,000-problem distribution

Track	Slots
AI Orientation and Computational Thinking	400
Python for AI	1400
Algorithms and Data Structures	1000
Mathematics for AI	1800
Data Skills	800
Machine Learning Foundations	1600
Advanced Machine Learning	700
Deep Learning	1100
Natural Language Processing and LLMs	500
Computer Vision	300
Reinforcement Learning	200
MLOps Responsible AI and Capstones	200
Total	**10000**

A manifest row is a planned production slot, not a finished problem. Publication requires automated correctness checks, technical review, editorial review, and pilot feedback.

Problem-type targets

multiple_choice: 2600; numeric_response: 1400; code_output: 1200; code_completion: 900; debugging: 800; coding_challenge: 1400; model_analysis: 700; mini_project: 500; case_study: 300; capstone_task: 200

Difficulty targets

novice: 2200; beginner: 2300; intermediate: 3000; advanced: 2000; expert: 500

4. Five core books

B01 - Python Foundations for Artificial Intelligence

Audience: Absolute beginners through early intermediate learners. Capstone: Create a tested Python data-analysis utility with a command-line interface and clear documentation.

Chapters: How Programs Think; Variables Types and Expressions; Control Flow; Functions and Decomposition; Collections and Data Modeling; Files Errors and Modules; Object-Oriented Design; Testing Debugging and Complexity; NumPy and Vectorized Thinking; Building a Small AI-Ready Data Tool

B02 - Mathematics for Machine Learning, Visually and Practically

Audience: Learners with basic school algebra. Capstone: Derive and implement linear regression with gradient descent using only Python and NumPy.

Chapters: Functions Graphs and Modeling; Vectors and Geometry; Matrices and Linear Systems; Linear Transformations and Eigen Ideas; Probability Foundations; Statistics Sampling and Uncertainty; Derivatives and Gradients; Optimization and Gradient Descent; Information Theory and Numerical Stability; Connecting Mathematics to Models

B03 - Algorithms, Data Structures, and AI Problem Solving

Audience: Python learners ready to develop algorithmic judgment. Capstone: Build and benchmark a search-and-ranking pipeline using multiple data structures and algorithms.

Chapters: Complexity and Measurement; Arrays Strings and Two-Pointer Patterns; Stacks Queues Hashing and Heaps; Recursion Trees and Traversals; Graphs and Network Reasoning; Sorting Searching and Selection; Greedy Algorithms; Dynamic Programming; Backtracking and Constraint Search; Algorithms Inside AI Systems

B04 - Machine Learning from First Principles

Audience: Learners who know Python and core AI mathematics. Capstone: Train, evaluate, interpret, and document a leakage-safe tabular ML system.

Chapters: The ML Workflow and Honest Baselines; Regression; Classification and Logistic Regression; Distance-Based and Probabilistic Models; Trees Forests and Ensembles; Support Vector Machines; Clustering and Dimensionality Reduction; Evaluation Validation and Imbalance; Feature Engineering Tuning and Interpretation; End-to-End Reproducible ML Project

B05 - Deep Learning, Generative AI, and Responsible Deployment

Audience: Intermediate learners ready to train neural models. Capstone: Deliver a small neural or retrieval-based AI product with evaluation, monitoring, limitations, and a model card.

Chapters: Neural Networks and Representation Learning; Backpropagation and Optimization; Training Stability and Generalization; Convolutional Networks and Vision; Sequences Attention and Transformers; Language Models and Embeddings; Generative Models and Retrieval; Evaluation Safety and Responsible AI; Serving Monitoring and Cost; Capstone: A Documented AI Product

5. Optional knowledge library

D01 - Advanced Python Patterns

Iterators, generators, decorators, context managers, typing, async concepts, profiling, and package design.

D02 - NumPy for High-Performance Numerical Work

Shapes, broadcasting, vectorization, memory layout, numerical stability, and benchmarking.

D03 - Pandas for Real Data

Indexing, joins, reshaping, time data, missing values, performance, and reproducible cleaning.

D04 - SQL for Data and AI

Relational modeling, joins, windows, CTEs, query plans, feature tables, and leakage prevention.

D05 - Linear Algebra Deep Dive

Vector spaces, bases, projections, decompositions, eigen concepts, SVD, and ML applications.

D06 - Probability and Statistics Deep Dive

Distributions, expectation, Bayesian reasoning, estimation, confidence, hypothesis tests, and calibration.

D07 - Calculus and Optimization Deep Dive

Multivariable derivatives, chain rule, constrained optimization, convexity, optimizers, and diagnostics.

D08 - Data Structures in Production

Implementation tradeoffs, caching, indexes, trees, graphs, approximate search, and memory behavior.

D09 - Algorithm Design Patterns

Divide-and-conquer, greedy proofs, dynamic programming, graph search, backtracking, randomized and approximation methods.

D10 - Exploratory Analysis and Data Visualization

Question-driven EDA, visual encodings, uncertainty, accessibility, dashboards, and misleading-chart prevention.

D11 - Data Preprocessing and Feature Engineering

Splits, leakage, encoding, scaling, missingness, outliers, feature selection, and pipelines.

D12 - Supervised Learning Deep Dive

Loss functions, bias-variance, regularization, linear models, trees, kernels, ensembles, and diagnostics.

D13 - Unsupervised Learning Deep Dive

Clustering, embeddings, manifold methods, anomaly detection, validation, and interpretation.

D14 - Time Series Forecasting

Temporal splits, baselines, stationarity, features, classical models, deep models, uncertainty, and drift.

D15 - Deep Learning with PyTorch

Tensors, autograd, modules, data loaders, training loops, mixed precision concepts, checkpoints, and debugging.

D16 - Computer Vision Systems

Image pipelines, CNNs, augmentation, transfer learning, detection, segmentation, evaluation, and deployment constraints.

D17 - Natural Language Processing

Tokenization, classical features, embeddings, sequence labeling, evaluation, multilingual issues, and error analysis.

D18 - Transformers, LLMs, and Retrieval

Attention, transformers, embeddings, RAG, tool use, structured output, evaluation, hallucination controls, and cost.

D19 - Reinforcement Learning

Bandits, MDPs, value methods, policy methods, offline concerns, reward design, simulation, and safe evaluation.

D20 - MLOps and Responsible AI Deployment

Versioning, tests, serving, monitoring, drift, privacy, security, fairness, model cards, incident response, and governance.

6. Product areas

Home

Continue learning, daily goals, mastery, due review, weak skills, and projects based on real events.

Learn

A prerequisite-based skill path with units, checkpoints, projects, placement, and visible mastery.

Practice

Adaptive review, mistakes, math, code, algorithms, ML analysis, and custom sessions.

Labs

A safe Python editor, Math Lab, Algorithm Visualizer, ML Lab, and guided notebooks.

Books and Knowledge

Searchable, bookmarkable, downloadable, progress-synced reading.

Content Studio

Versioned authoring, validation, review, publishing, rollback, and audit history.

7. Orange design direction

Warm orange and cream surfaces, strong dark ink, professional icons, generous whitespace, accessible focus, restrained motion, and serious editor surfaces. The result remains CodeQuest and does not copy another product's assets or exact layout.

Token	Value	Use
Primary	#E56A3A	Actions and active path
Pressed	#C9542A	Hover and pressed
Canvas	#FBF5EE	Warm background
Surface	#FFFDFC	Cards and reading
Ink	#271F1B	Primary text
Border	#E8D8CE	Dividers

8. Implementation sequence

00. Read MASTER_AGENT_PROMPT.md and every source-of-truth specification.

01. Run AGENT_PROMPTS/00_AUDIT_EXISTING_APP.md. Stop after reporting the audit.

02. Run 01_MIGRATION_ARCHITECTURE.md. Obtain owner approval for irreversible or high-risk choices.

03. Run 02_DATABASE_AND_CONTENT_MODELS.md with additive migrations and rollback tests.

04. Run 03_ORANGE_UI_AND_NAVIGATION.md and present the design for approval before broad implementation.

05. Run 04_LEARNING_AND_MASTERY_ENGINE.md.

06. Run 05_EDITORS_AND_LABS.md with isolated execution and quotas.

07. Run 06_CONTENT_STUDIO_AND_IMPORT_PIPELINE.md.

08. Run 07_GENERATE_VALIDATE_CONTENT.md in controlled batches. A planned manifest row is not a completed problem.

09. Run 08_BOOKS_AND_KNOWLEDGE_LIBRARY.md.

10. Run 09_SECURITY_ACCESSIBILITY_ANALYTICS.md.

11. Run 10_TEST_MIGRATE_AND_DEPLOY.md.

12. Run 11_FINAL_ACCEPTANCE.md and do not declare completion until every blocking check passes.

9. Definition of done

Completion requires migrated users and progress, real learning flows, safe code execution, validated content, exact book page counts, optional deep dives, responsive and accessible UI, monitoring, tests, production build, deployment health, and a tested rollback path.